Thank you for the introduction!

The Simulation of Quantum-Many body systems is a very challenging problem. A general many-body wavefunction that describes a system with N subsystems can be written as follows. We sum over basis states, which are a product of local basis states of the subsystems. The complex amplitude of each basis vector is stored in this complex number Psi. To describe the full wavefunction we therefore need d^N parameters, where N is the system size. Because of this exponential scaling it is very hard to handle the wavefunction computationally, and therefore we need approximations. 0:40

So this is the problem we want to solve, which brings me to the contents of my talk. I will start by introducing Matrix Product States (MPS), which have had great success in tackling this problem in one-dimensional systems. Next, I will talk about isometric Tensor Product States (isoTPS), which generalize the canonical form of MPS to two and higher dimensions. In the third section of my talk I will discuss the work of my thesis, which was implementing an alternative canonical form for isoTPS. I will discuss how this canonical form is defined and how certain algorithms can be implemented. I will then show benchmarks of this method on the Transverse Field Ising Model. 0:40

Before I start with MPS, I want to give a quick general introduction to tensor networks. We define a tensor of rank n to be an n-dimensional array of complex numbers T. It is convenient to introduce a diagrammatic notation. In this notation, we draw tensors as shapes, and indices as lines emerging from these shapes. A few simple examples: A vector has only one index, so we can draw it as just a circle with a singular index sticking out. A matrix has two indices which we call I and j, and as a last example I show here a general tensor of rank n. 0:40

I want to also introduce a linear operation on tensors called index contraction. An index contraction of two tensors is performed by multiplying the two tensors, summing over certain connected indices. A simple example would be the scalar product of two vectors. Both vectors have a single index alpha, over which we sum here. The result is a scalar. In the diagrammatic notation, index contractions are denoted by connecting the bonds that are contracted. Vectors have one index, so we can just connect the two legs of A and B to form the scalar product. Similarly, the matrix product of two matrices is an index contraction, and can also be drawn as a tensor diagram. 0:40

A tensor network is then just defined as a set of tensors, that are contracted in a specific way. For example, this is a tensor network made out of three tensors that are contracted in a triangle shape. 0:20

Matrix product states are a powerful Ansatz for one-dimensional quantum states, for example spins on a chain. I write here again the many-body wavefunction. Note that we can interpret the amplitude Psi as a tensor of rank N. The idea of MPS is now to write this large tensor as a contraction of many smaller tensors T of rank 3. If we additionally restrict the bond dimension between these tensors to a maximum value of chi, we have then reduced the number of parameters from d^N to N chi^2 d, which is linear in system size. Of course this is an approximation. However, as it turns out, ground states of local, gapped Hamiltonians have an entanglement structure that follows an area law, which means that when cutting the chain into two parts, the entanglement between the left and right part is bounded by a constant. Remarkably, these states are exactly those that can be represented efficiently by the MPS ansatz. 1:00 -> 4:00

A powerful feature of MPS is the existence of a canonical form which greatly speeds up many algorithms. To discuss this canonical form, we must first introduce isometries. An isometry is a m x n matrix W with n >= m, which satisfies the so called isometry condition: The adjoint of W multiplied with W must be equal to the identity. Multiplying the other way must give a projector P. In tensor diagrams we draw this by decorating the legs of the tensor with arrows. As a convention we draw the larger dimension with incoming arrows, and the smaller dimension with outgoing arrows. The isometry condition can then be visualized as follows: Contracting W with its complex conjugate along the bond of dimension n gives identity, and contracting the other way gives a projector. 0:50

An isometric tensor is then a tensor that can be transformed into an isometry by grouping all the legs with incoming and outgoing arrows respectively. So if we contract all legs with incoming arrows, we obtain identity. 0:15

Every MPS can easily and exactly be brought into a canonical form. In this canonical form, all tensors are isometries except for the orthogonality center tensor, which we draw in orange. All arrows point to the orthogonality center. The orthogonality center can be easily moved to the left and right through QR-decompositions. We call tensors left of the orthogonality center A and tensors to the right B. The A tensors fulfil a left-isometry condition, when contracting with the complex conjugate from the left, they give identity. Similarly, the B-tensors fulfil a right-isometry condition. 0:40

This allows us to simplify certain algorithms. As an example i want to show the computation of a local expectation value. Let us assume that we want to compute the expectation value of an operator that acts on the site that is represented by the orthogonality center. We can interpret the operator as a tensor of rank two, and computing the expectation value can then be drawn as this tensor network, where the upper part is the ket and the lower part is the bra and the operator is sandwiched in between. Since we are in the canonical form, we can successively use the isometry condition at the left and right boundaries to reduce the expectation value to a contraction of only 3 tensors! 0:40 -> 6:25

With MPS one can also perform time evolution with the Time Evolving Block Decimation algorithm, or TEBD for short. The Schrödinger equation tells us that to evolve a state by a time Delta t, we need to apply the time evolution operator U, which is the exponential of -i*Delta t times the Hamiltonian  Let us assume that the Hamiltonian can be split into terms acting only on neighboring sites. We can then split the Hamiltonian into two terms, one acting only on even and one acting only on odd bonds. We can then approximate this exponential by a so-called Suzuki-Trotter decomposition with an error of Delta t^2. This is then the approximate time evolution operator we apply to the state. Because all operators acting on even bonds commute with each other, this exponential factorizes into bond operators acting only on neighboring bonds, and the same holds true for the odd terms. 1:00

If we now want to evolve the MPS by one time step, we need to apply these operators to the MPS. We can draw this as a tensor network as follows: Here we have the MPS, and we act first with the bond operators on all even bonds and then on all odd bonds. Now, we need to approximate this network with an updated MPS. This can be done by absorbing the bond operators into the MPS one after the other.

A single bond operator can be absorbed by contracting it with the two MPS tensors into a tensor theta, and then splitting this tensor again with a truncated SVD. 0:35 -> 8:00

Next, I introduce isometric Tensor Product States (isoTPS), which generalize the canonical form of MPS to two and higher dimensions. First of all, the natural generalization of MPS to higher dimensions is known as Projected entangled pair states (PEPS), where we now write the wavefunction of a system on a 2D square lattice as the contraction C of tensors as shown in this tensor network. We call the bond dimension of the legs drawn in black D. The problem with this approach is that it is not possible to transform the network into a canonical form without error because of the presence of closed loops. Because of this, algorithms defined for PEPS are often very expensive and also instable. For example, a full TEBD update scales with the bond dimension D to the power of 10. 0:50

Isometric Tensor Product States are then defined as follows. The tensors are isometrized in a way that all arrows point towards a special row and column which is called the orthogonality hypersurface and is here drawn in red. Along the orthogonality hypersurface, all arrows point to a singular orthogonality center, which we again draw in orange. Now, similar to MPS, we call columns left of the orthogonality hypersurface A and columns right of the orthogonality hypersurface B, while we call the orthogonality hypersurface itself Lambda. In contrast to MPS, a PEPS can not be transformed into an isoTPS without error. We again call the maximum bond dimension along the black bonds D. In practice one achieves good results when increasing the bond dimension along the orthogonality hypersurface to chi = f*D, with an integer f. 1:00

Most algorithms defined on isoTPS need to be able to move around the orthogonality center. Moving the orthogonality center along the orthogonality hypersurface is easy, and can be done via simple QR-decompositions.

Moving the orthogonality hypersurface to the left or to the right is a harder problem. The orthogonality hypersurface can be moved one column to the right by first splitting it into two columns A and Lambda, and then absorbing Lambda to the right into the column B. This second step is simply the problem of multiplying a Matrix Product State with a Matrix Product Operator, for which efficient algorithms exist. The first step is harder. One solution is to variationally optimize the two columns by sweeping over all tensors and performing local updates. However it is found that an iterative procedure, the so-called Moses-Move, is more efficient while producing a solution that is almost as good as the variational one. 1:00 -> 10:50

The Moses-Move is performed as follows: We start at the bottom of the orthogonality column, and split the orthogonality center into three tensors using a tripartite decomposition. We then absorb the orange tensor into the next upper tensor. We now repeat this procedure, again splitting into three tensors and so on, until the top of the column is reached. The question is now how this tripartite decomposition can be performed. 0:30

A tripartite decomposition can be performed as follows: First, the tensor W is split into two tensors Q and theta via a singular value decomposition, where the bonds are also truncated to D^2. Next it is crucial to note that there is a gauge degree of freedom on the connecting bonds: we can insert a unitary and its adjoint. After absorbing U into theta and Udagger into Q, we then split theta via another SVD into two tensors B and C, finalizing the tripartite decomposition. The unitary U can now be chosen in such a way that it minimizes the error of the final split in the last step. A good choice is a unitary that reduces the entanglement between the subsystems represented by the upper and lower indices of theta. This unitary is therefore called the disentangling unitary. We will discuss how to find a good disentangling unitary later. 1:00 -> 12:20

I also want to give two examples of algorithms that have been implemented on isoTPS. First, TEBD can be implemented very similar to MPS. The algorithm is much faster than for PEPS,

reducing the computational complexity from D^10 to D^7! Second, one can implement a DMRG algorithm for performing ground state search, which scales as D^12, which is the same scaling as PEPS. However the canonical form transforms the generalized eigenvalue problem into a standard one, which is much more stable computationally. 0:30

In my thesis, we implemented an alternative canonical form for isoTPS. The canonical form is defined as follows: First, a PEPS is rotated by 45 degrees as shown here. We then introduce so-called auxiliary tensors, which we draw in red, and which make up the orthogonality hypersurface. The orthogonality hypersurface is placed in between two rows of PEPS tensors. Note that the auxiliary tensors do not carry any physical degrees of freedom. Again, all arrows point towards the orthogonality hypersurface and towards the orthogonality center. In the dashed lines i draw a unit cell, which now includes two sites. We again call the bond dimension of the black bonds D and the bond dimension of the red bonds is set to chi = f*D. 0:50

In analogy to MPS and isoTPS, one can easily evaluate local expectation values. Lets say we want to compute the expectation value of an operator acting only on two neighboring sites. To compute the expectation value, we contract the isoTPS with its complex conjugate, sandwiching the operator in between. Now we can use the isometry conditions starting from the boundary, reducing contractions to identity, until we end up with a contraction of only 11 tensors. 0:30

As in the original isoTPS, most algorithms need to be able to move around the orthogonality center. Again, moving the orthogonality center along the orthogonality hypersurface is easy and just requires QR-decompositions. The problem arises again when we want to move the hypersurface to the left or to the right. This can be done by starting with the orthogonality center at the bottom of the orthogonality hypersurface. Now, two auxiliary tensors are pulled through the physical tensor to the other side. Then we move up the orthogonality center and again pull two tensors through to the other side, until we reach the top. 0:35 ->14:45

The main difficulty is the process of pulling the auxiliary tensors through the physical tensor, which we draw again here. We call this a Yang-Baxter move, because of the visual similarity to the Yang-Baxter equation. We call the tensors before this move T, W_1 and W_2, and denote the tensors after the move by primed tensors. The error of the YB-move can be written as the norm of Psi – Psi^prime. If we impose the additional condition that both states are normalized, we can rewrite this error as the square root of 2 – 2 times the real part of the overlap. 0:35

The problem of finding a good YB move can therefore be written as a constrained optimization problem: The best tensors T^prime, W_1^prime and W_2^prime are the ones that maximise the real part of the overlap, under the constraint that T^prime and W_1^prime are isometries and W_2^prime is normalized to one. In my thesis, I looked at two algorithms: A variational optimization with local updates and a tripartite decomposition similar to the one used in the MM. 0:35

Let's start with the first of the two algorithms, the variational optimization. The real part of the overlap can be written as the following tensor network, where we have again used the isometry condition to turn a contraction of the full networks to a contraction of only a few tensors. Around the orthogonality center. One can now maximize this overlap by sweeping over the three primed tensors and optimizing them iteratively. For example, lets start by fixing W_1^\prime and W_2^\prime and only vary T_prime. In this case the optimization problem is known as the orthogonal Procrustes problem and permits a closed form solution. We then iterate over the three tensors and optimize them locally with these closed form solutions. A similar algorithm was used by Evenbly and Vidal in the context of the Multiscale Entanglement Renormalization Ansatz. 0:45 -> 16:50

Lets now qualitatively discuss how the algorithm performs. I plot here the approximation error against the number of iterations. As you can see, the algorithm converges only very slowly, It is not converged even after 10'000 iterations. 0:20

The second algorithm for the YB move is a tripartite decomposition similar to the MM. We start by contracting the three tensors into a big tensor Psi. This tensor is then split horizontally via a truncated SVD, and we truncate the purple leg to a bond dimension of D^2. In the next step, we split the connecting bonds into two bonds of bond dimension D. As in the MM, we can now insert a disentangling unitary U and its adjoint. This can be used to minimize the error of the next step, where we split theta vertically, finishing the YB-move. The disentangling is crucial for a good tripartite decomposition, so lets talk about it in more detail. 0:45

First a bit of notation: The unitary U is contracted with theta, forming a tensor theta tilde. We group the upper and lower legs into these purple legs with bond dimension chi*D. The tensor is then split along the vertical direction with an SVD into tensors X, S, and Y. 0:25

So how do we choose a good disentangling unitary? The idea is to choose U such that it minimizes a given cost function. We looked at two cost functions, that were also discussed for the MM. The first one is simply the truncation error of truncating the last SVD to a bond dimension of chi. The second cost function is the Renyi-entropy. Its given by 1/(1- a parameter alpha) times the logarithm of the trace of the density matrix to the power of alpha. It is a measurement of entanglement. The density matrix is obtained by tracing out one side of the theta tilde tensor. Therefore, we can write this Trace of rho to the power of alpha as a sum over the singular values to the power of two alpha. 1:00

If we set alpha = 2, the renyi entropy takes on a particularly simple form. Its then just given as – the trace of rho^2. The trace of rho^2 can actually be easily written as this tensor network. And what we can now do, we can again use the Evenbly-Vidal algorithm to maximize this trace, which is equivalent to minimizing the renyi-entropy. So we fix all tensors except for one U, and then the problem permits a closed form solution. We then update U and repeat this procedure. Contracting this network which must be done once per iteration, scales as D^9. 0:40

This seems quite expensive, but as it turns out, this algorithm converges really fast in practice. Here this is qualitatively shown, and as you can see here this converges after roughly 20 iterations, where the first algorithm wasn't converged even at 10'000 iterations. 0:20

Ok, so this was for the renyi-entropy with alpha = 2, but what if we want to use a different alpha or use the truncation error directly as cost function? In that case, the cost function cannot easily be written as a tensor network and we cannot use the same optimization method. However, there exists a really powerful framework called Riemannian optimization of matrix manifolds. We want to optimize over the set of all unitary matrices. More generally, the set of all complex n times m isometries is called the Stiefel manifold. The idea of Riemannian optimization is to optimize a cost function while staying on this manifold. Lets say our current best guess for the unitary is given by some point on the manifold, lets call it U_k. One can now compute a update direction xi_k which has to be projected to the tangent space of U_k which we draw here in green. This update vector can for example simply be the negative gradient of the cost function. We want to now move in the update direction, but if we do that, we would leave the manifold. To solve this problem one introduces a so-called retraction R, which we draw here in gray, which can be thought of as moving in the direction xi_k while staying on the manifold. We can then move along this path to obtain a new iterate U_{k+1} 1:30 -> 21:50

For the cost functions we discussed the gradient can be computed analytically, here is for example the gradient of the truncation error cost function. Computing the gradient also costs D^9. This high cost arises from the SVD and from contractions involving the tensors X and Y. What we found is that if we instead approximate the gradient by only compute an approximate SVD, directly truncating this bond dimension, we can achieve similar results at much lower costs.

We can perform this approximate SVD as follows: First, we split theta tilde into Q and R via an approximate QR-decomposition. This approximate QR-decomposition can be performed by again using an Evenbly-Vidal style optimization algorithm, computing this overlap and alternatingly optimizing Q and R. When this is converged, we then split the tensor R via a normal SVD. This iteration here converges very quickly, in practice, this takes less than 5 iterations to converge. Approximating the gradients with this technique we are able to reduce the cost from D^9 to D^8 + N_qr*D^7. 1:20 -> 23:10

In this plot, I show the truncation error cost function plotted against the time. I used the Trust-region method (TRM), which is a Riemannian optimization method of second order. As you can see, the approximate version of the algorithm performs similar to the exact version while being about an order of magnitude faster. 0:20

I now want to introduce one more algorithm, which is the generalization of TEBD to isoTPS in the alternative canonical form. First of all, lets look at how a local bond operator can be applied. We want to apply the bond operator to two neighboring tensors, resulting in an updated state psi^prime. We can now again write this as an optimization problem, optimizing this overlap, with now the bond operator sandwiched in between. To solve this, we again use the Evenbly-Vidal style algorithm, iteratively optimizing the bottom five tensors. Because the time step Delta t is small, this operator here is close to identity. Therefore, a good initialization for the updated tensors are just the original tensors, and the algorithm converges very quickly after only a few iterations. 0:45

To globally apply now a TEBD update of first order, we start in the left-most column and apply all bond operators that act on this column. We then move two columns to the right and again apply all bond operators. We repeat this until the right-most column is reached. On the way back, we then apply the bond operators on all odd columns, concluding the first order update. We can very easily improve this update to second order by symmetrizing the first-order Trotter decomposition and applying the operators along a chain. This then looks like this: We again start at the left-most column, but now apply the operators in a chain. We move one column to the right and again apply along a chain. We repeat this until we hit the right edge. On the way back, we again apply operators on each column, but in the reverse order. Arriving back at the left edge, we have then applied a second order TEBD step with the error scaling as the third power of the time step. 1:00 -> 25:15

In order to benchmark this method we looked at the Transverse Field Ising model. The model is a spin-1/2 lattice model. The Hamiltonian is made up of two terms, one interaction term and a transverse field in z-directoin. At zero temperature, the model exhibits a quantum phase transition, which happens at a critical field of roughly 3.04 on the 2D square lattice. We start by using imaginary time evolution to find the ground state on the models. The error of TEBD is made up of three parts: The trotterization error, the error from applying the local bond operators and the error of performing the YB move. The first two errors get smaller when choosing a smaller time step, but the YB move error is not directly affected by the time step. However, for smaller time steps we need to compute more TEBD steps for reaching the same time, and this also

means more YB errors. Therefore it is clear that there exists a sweet spot, where the two parts of the errors are roughly the same, and this is the regime where the overall error will be minimized. 1:10 -> 26:25

This simulation was performed on a 4x4 diagonal square lattice with 32 spins. We look at 4 different bond dimensions, 2, 4, and 6 and use second order TEBD. For comparison, we performed a DMRG reference simulation with tenpy, using an MPS that is snaked through the lattice. First, lets compare the different methods for the YB move. In the left-most figure, a simple SVD was used. We can already see, that there exists a minimum error for some optimal time step Delta t. In the second figure we use the iterative Evenbly-Vidal style algorithm, which performs about the same. The reason for this is its slow convergence. Minimizing the Renyi-2 entropy works really well compared to the other algorithms, and on top of that is very fast. For this reason, we use this disentangling as an initialization for the following methods. 0:45

Interestingly, choosing the truncation error as a cost function performs a bit worse than the Renyi-2 entropy. The reason for this is that while the error of a single YB move is smaller, the error of moving a complete column is actually larger when directly disentangling the truncation error. Iin that way the Renyi-entropy is a more physical cost function, which is able to represent the full state better. Notably, the approximate TRM perform almost as good as the exact TRM while being much faster. 0:35

In our testing, the best method was found to be the Riemannian optimization of the Renyi-entropy with alpha = ½, which reaches relative energy errors of well below 10^{-5}. In the following, we will therefore use the approximate TRM, optimizing the Renyi entropy with alpha = 0.5. Again, we see that the approximate version of the algorithm basically performs as good as the exact version. 0:30 ->28:15

Its also interesting to compare TEBD of first order to TEBD of second order. As you can see, TEBD2 can obtain an error that is about an order of magnitude smaller. Note the x-axis: we can go to much larger time steps when using TEBD2 because of the smaller trotter error. This way, we perform less YB moves per unit time, and this leads to a lower error. 0:30

Let us now go to larger lattices. When increasing the system size from a 4x4 lattice to a 7x7 lattice, the error becomes larger, but seems to converge to a constant relative error when going to larger and larger systems. 0:20

To investigate this further, we performed a ground state search up to a system size of 20x20 which corresponds to 800 spins in total. First, let's look at the DMRG reference simulation. We plot the energy density against the linear system size. We would expect this energy density to approach a constant for infinite system size, since the finite size effects become less and less important. However, as you can see, the reference simulation diverges at some point. This happens earlier for smaller maximum bond dimensions chi. Here the black line is obtained by extrapolating the bond dimension to infinity. The reason for this divergence is that the MPS fails to properly represent the 2D area law at some point. In contrast, the isoTPS simulation doesn't show this divergence and really seems to approach a constant energy density. This is numerical evidence that the ansatz is able to correctly represent the 2D area law entanglement structure of the TFI model ground state! 1:00 ->30:05 😵

Finally, we also performed real-time evolution after a global quench. For this, we initialized the system with all spins pointing up on an 8x8 square lattice, and then performed a time evolution. Here we plot the sigma_z expectation value of a spin in the center of the lattice against time. This simulation was performed at the critical field, and we therefore expect the entanglement to grow

very quickly with time, making this a hard problem. As you can see here, the method still has problems with real-time evolution, diverging really quickly. The error of the YB move is the main problem. As a potential solution, one could variationally optimize the tensors after shifting a column completely, this was also done on the original isoTPS after the MM and found to be especially important when doing real-time evolution. Alternatively, one could try to find a better procedure for performing the YB move. 0:50 -> 30:55

This brings me to a short summary of my talk. We have introduced a new canonical form for isoTPS. The YB move is a bit more expensive than the MM, scaling as D^8 instead of D^7. One advantage of the YB-isoTPS is that it is very easily generalizable to other lattices. For example, I also generalized it to the honeycomb lattice as shown here, and with this we were also able to find ground states of the TFI model. Also, more involved algorithms like DMRG anyways scale with high powers like D^12, making it less important if moving the ortho surface scales with D^9 or D^8.

As an outlook: It would be interesting to implement a DMRG like energy minimization algorithm and compare it to the original isoTPS. Also it would be nice to generalize this method to further lattices, for example the Kagome lattice. Finally, the algorithm would greatly benefit from porting it to Graphics Processing Units (GPUs), as they can significantly speed up tensor contractions and decompositions, while also increasing the power efficiency.

This concludes my talk; id be happy to answer some questions! 1:15 -> 32:05